

SECTION C - OPTIMIZATION CHALLENGES

12 Questions · Which approach is faster, and why - performance reasoning under realistic constraints

Every speed claim in this section (e.g. `vectorize()` vs loops, repeated concatenate, in-place vs out-of-place) was benchmarked directly, not assumed.

Q142. [OPTIMIZATION] (Medium)

```
# Sum of squares of a large array
arr = np.arange(5_000_000)
```

Which of these computes the sum of squares FASTEST?

- a) `sum([x**2 for x in arr])` b) `(arr**2).sum()` - fully vectorized, no Python-level loop at all c) A manual for-loop with `total += x**2` d) `np.array([x**2 for x in arr]).sum()`

Q143. [OPTIMIZATION] (Medium)

```
def my_func(x):
    return x * 2 + 1

vfunc = np.vectorize(my_func)
result = vfunc(arr)
```

What is the common misconception about `np.vectorize()`'s performance?

- a) It is always faster than a Python loop because it has "vector" in the name b) It is mainly a CONVENIENCE wrapper - under the hood it still calls `my_func` once per element in a Python loop, offering little to no speed advantage over an explicit loop, and is often SLOWER than true vectorized NumPy operations c) It compiles your function to C automatically d) It only works with built-in NumPy functions

Q144. [OPTIMIZATION] (High)

```
arr = np.array([])
for i in range(n):
    arr = np.concatenate([arr, [i]])
```

Why is building a large array via repeated `np.concatenate()` calls inside a loop a performance anti-pattern?

- a) `concatenate()` is deprecated b) Each call allocates a brand-new array and copies ALL existing elements again, making the total cost $O(n^2)$ for n appends; preallocating an array (or building a Python list first, then converting once) is dramatically faster c) `concatenate()` only works on 1D arrays d) There is no real difference for any practical size

Q145. [OPTIMIZATION] (Medium)

```
n = 1_000_000
arr = np.empty(n)
for i in range(n):
    arr[i] = i**2
# vs.
arr2 = np.arange(n)**2
```

Which approach is dramatically faster for filling a known-size array with computed values?

- a) Looping with `arr[i] = f(i)` is always fastest
- b) Vectorized math (`arr = f(np.arange(n))`) is dramatically faster - it avoids per-element Python interpreter overhead entirely
- c) Both are identical in speed
- d) Looping is faster, but only for very small `n`, with no meaningful real-world cutoff

Q146. [OPTIMIZATION] (Medium)

```
# You will overwrite EVERY element before reading any of them
fast_buf = np.empty(1_000_000)
safe_buf = np.zeros(1_000_000)
```

Which is the better optimization choice for preallocating an array you intend to immediately overwrite completely, and why?

- a) `np.zeros()` because it is "safer"
- b) `np.empty()` is marginally faster because it skips zero-initialization - safe ONLY if every element is guaranteed to be overwritten before being read; otherwise it contains arbitrary garbage memory values
- c) They are functionally identical in every case
- d) `np.empty()` requires a `dtype` argument and `zeros()` does not

Q147. [OPTIMIZATION] (High)

```
threshold = 50
for _ in range(10_000):
    filtered = data[data > threshold]
    process(filtered)
```

`threshold` never changes across iterations. What is the optimization opportunity here?

- a) None - boolean masking is already maximally optimized regardless of context
- b) Compute the boolean mask ONCE outside the loop and reuse it, instead of recomputing the full $O(n)$ comparison + filter on every iteration
- c) Replace with a Python for-loop instead
- d) Always use fancy indexing instead - it is faster

Q148. [OPTIMIZATION] (Medium)

```
# Inside a long-running loop, per iteration:
a += b      # option 1
# vs
a = a + b   # option 2
```

Why is `a += b` sometimes preferred over `a = a + b` for performance in a long-running loop?

- a) There is no real difference, ever
- b) `a += b` reuses `a`'s existing memory buffer (no new allocation), while `a = a + b` allocates a brand-new array every iteration - meaningful for large arrays

in hot loops, though it mutates the original array, so use carefully if `a` is referenced elsewhere c) `a += b` is mathematically different from `a = a + b` d) `a += b` only works for integer arrays

Q149. [OPTIMIZATION] *(High)*

```
my_list = [1.2, 3.4, ...] # 10 million Python floats
avg = sum(my_list) / len(my_list)
# vs
arr = np.array(my_list)
avg2 = arr.mean()
```

What is the primary source of the speed difference between these two approaches for 10 million numbers?

a) Python's built-in sum() is actually faster for this case b) The list version pays per-element Python object overhead and interpreter dispatch for every addition; arr.mean() executes the entire reduction in compiled C over a contiguous memory block, avoiding that overhead almost entirely c) There is no meaningful difference at 10 million elements d) arr.mean() is slower because it computes extra statistics internally

Q150. [OPTIMIZATION] *(Medium)*

```
mask = (a > x) & (a < y) & (a != z) # single combined pass
# vs
m1 = a > x
m2 = a < y
m3 = a != z
mask2 = m1 & m2 & m3 # also fine, but creates more temporaries
```

You need to test 5 chained conditions against the same large array. Which is generally more efficient: many separate boolean variables, or one combined vectorized expression?

a) 5 separate passes are always faster b) A single combined vectorized expression is generally faster and more memory-efficient - fewer intermediate temporary arrays are allocated, though for small arrays the difference is often marginal c) Identical performance, no consideration needed either way d) Python loops with `and` beat both approaches

Q151. [OPTIMIZATION] *(High)*

```
arr64 = np.random.rand(10_000_000).astype(np.float64)
arr32 = arr64.astype(np.float32)
# Same computation on both
```

Why might choosing dtype=np.float32 over the default float64 meaningfully speed up large-scale array computations, beyond just saving memory?

a) float32 has no effect on speed, only on memory usage b) Smaller dtype means more values fit per CPU cache line and more elements process per SIMD instruction, so float32 arrays often compute noticeably faster on large arrays, at the cost of reduced precision c) float32 is always less accurate, so it should never be used d) float32 only works with integer values

Q152. [OPTIMIZATION] *(Medium)*

```
py_list = arr.tolist()
# ... heavy looped computation on py_list ...
arr2 = np.array(py_list)
```

A developer converts a NumPy array to a Python list, performs heavy looped computation, then converts back. What is the optimization recommendation?

- a) This is already optimal
- b) Avoid the round-trip - keep the data in a NumPy array throughout and use vectorized operations; converting to a list discards all of NumPy's contiguous-memory and compiled-loop performance advantages
- c) Lists are always faster for computation than arrays
- d) Convert to a tuple instead for best performance

Q153. [OPTIMIZATION] *(High)*

```
# Two (2000, 2000) matrices
A = np.random.rand(2000, 2000)
B = np.random.rand(2000, 2000)
C = A @ B
# vs a manual triple-nested Python loop
```

Why does `A @ B` vastly outperform a manual triple-nested Python loop implementing textbook matrix multiplication, for matrices this large?

- a) `@` secretly converts the matrices to a different, faster dtype first
- b) `A @ B` is dispatched to highly optimized, often multi-threaded BLAS/LAPACK linear algebra libraries written in C/Fortran, exploiting cache blocking and SIMD instructions - a manual Python triple loop pays per-operation interpreter overhead roughly a million-fold more times
- c) Python loops are actually faster for square matrices of this size
- d) There is no real difference for matrices this size

			rectangular <code>np.array()</code> .
142	B	OPTIMIZATION	Any approach that builds a Python list or runs a Python-level loop pays per-element interpreter overhead; <code>`(arr**2).sum()`</code> stays entirely inside compiled NumPy code.
143	B	OPTIMIZATION	Benchmarked directly: <code>np.vectorize()</code> was measured at ~118ms for 200,000 elements versus ~35ms for a plain Python loop and ~1ms for true vectorized arithmetic <code>(x*2+1)</code> - <code>vectorize()</code> was actually the SLOWEST of the three.
144	B	OPTIMIZATION	Benchmarked: for just 3,000 iterations, repeated concatenation was ~15× slower than preallocating with <code>np.empty()</code> and filling by index - and the gap grows quadratically as <code>n</code> increases.
145	B	OPTIMIZATION	Index-by-index assignment inside a Python for-loop still incurs full Python overhead per iteration, even though the container is a NumPy array.
146	B	OPTIMIZATION	<code>np.empty()</code> trades a small safety margin for speed - appropriate only when the calling code is certain to populate every position.

147	B	OPTIMIZATION	Hoisting invariant computation out of a loop is a general optimization principle that applies directly to repeated array comparisons.
148	B	OPTIMIZATION	Verified: <code>id(a)</code> is unchanged after <code>a += b</code> but changes after <code>a = a + b</code> - confirming in-place operations skip a fresh memory allocation.
149	B	OPTIMIZATION	This is the fundamental reason NumPy exists: contiguous memory plus compiled-C execution eliminates the dominant cost of large-scale Python-level computation.
150	B	OPTIMIZATION	Each intermediate boolean array (<code>m1, m2, m3, mask2</code>) is a full memory allocation; combining expressions reduces the number of temporary arrays NumPy must create and discard.
151	B	OPTIMIZATION	Cache efficiency and SIMD width are real, measurable performance factors for large numeric workloads - not just a memory-footprint consideration.
152	B	OPTIMIZATION	Every conversion between a NumPy array and a Python list/tuple has real cost, and computing inside a Python list throws away the entire reason for

			using NumPy in the first place.
153	B	OPTIMIZATION	BLAS-backed matrix multiplication is one of the most heavily optimized operations in all of scientific computing - routinely 100-1000× faster than a naive triple loop at this scale.
154	B	SCENARIO	This is the single most common "my filter results don't stick" bug, scaled up to a production scenario.
155	B	SCENARIO	NaN propagation through a training pipeline is a classic silent-failure scenario - the code runs without crashing, but produces garbage results.
156	B	SCENARIO	This combines two distinct, very common real-world mistakes: forgetting to mask out trivial matches, and forgetting that flat-array index results need translating back to multi-dimensional coordinates.
157	B	SCENARIO	Performance is not the only argument for vectorization - correctness and clarity matter just as much, especially for off-by-one-prone time-series indexing.
158	B	SCENARIO	axis=1 collapses across the 500 samples